Flight Delay Forecasting

Kevin Brugnera†, Sara Pasquato†, Libero Pollini†

***Abstract*—Flight delay forecasting is a relevant challenge in modern air transportation systems, as accurate predictions can support more efficient airline operations, airport management, and passenger services. However, modeling flight delays remains challenging due to the complex interaction between heterogeneous static flight attributes and the temporal propagation of delays across consecutive flights operated by the same aircraft. This work investigates a multimodal deep learning approach for flight delay prediction using the Aeolus multi-structural flight delay dataset. The proposed architecture combines two complementary branches: a tabular branch that analyzes static flight information and a sequential branch that models delay propagation along aircraft flight chains. In the tabular branch, LightGBM is employed for feature analysis and selection, while a Multi-Layer Perceptron (MLP) learns a compact representation of the selected flight attributes from Aeolus data. The sequential component is based on a Long Short-Term Memory (LSTM) network, which captures temporal dependencies within flight chains. The learned MLP embedding is then fused with the LSTM representation to integrate static contextual information into the final prediction model. The results show that combining tabular and sequential information in a fused model does not provide significant performance improvements, while slightly degrading performance and increasing computational costs. This suggests that, for the considered dataset, the sequential information captured by the LSTM already provides a strong predictive representation, and the additional static information contributes only marginal benefits.**

***Index Terms*—Flight Delay Forecasting, Long Short-Term Memory, LightGBM, Multi-Layer Perceptron, Multimodal Learning, Feature Fusion.**

I. Introduction

Flight delay prediction is a task of significant economic and environmental relevance: in 2022 alone, flight disruptions in the United States generated estimated losses of $30–34 billion and roughly 3 million additional tons of CO_2 emissions [1]. Accurate, timely delay forecasting can support airlines, airports and passengers in mitigating these costs.

Delays originate from stochastic factors – weather, air traffic control interventions, technical faults – but propagate through complex spatio-temporal dynamics: a delayed flight can affect gate assignment, crew rotation and airspace congestion for subsequent flights operated by the same aircraft [2]. Existing public datasets and models have historically addressed this problem through purely tabular, flight-level representations, largely ignoring this sequential propagation. Among the works reviewed here, LATTICE [3] is the closest architectural precedent, combining a sequential and a static branch for arrival delay classification; however, it operates at a much smaller scale (a single airport, approx. 15,000 flights over 6 weeks, using ADS-B trajectories) and without a temporal evaluation protocol. Among the works reviewed in Section II (McCarthy et al. [4], Sun et al. [5], TrajCNN [6], DenseNet-LSTM-BLS [7], Desai et al. [8] and Jha et al. [9]) none combines tabular and sequential representations end-to-end at the scale of a large, multi-year, publicly available dataset with a rigorous temporal split.

This work addresses flight delay prediction on Aeolus [2], a recently released large-scale multimodal benchmark (54.67M flights, 320 airports, 9 years). We design and evaluate: (i) a tabular branch, comparing gradient-boosted trees (LightGBM, with and without hyperparameter tuning) against a neural network with learned categorical embeddings (MLP); (ii) a sequential branch, an LSTM operating on the flight chain modality of Aeolus (i.e., the sequence of flights operated by the same aircraft); and (iii) their fusion into a single end-to-end architecture, inspired by the dual-branch design of LATTICE [3]. The tabular branch addresses arrival-delay prediction as binary classification (delay $>$15 minutes), consistent with Aeolus's own tabular benchmark; the sequential branch and the fused architecture instead perform regression on delay duration (minutes) for both arrival and departure delay. As a result, the standalone tabular results and the sequential/fusion results are evaluated with different metrics (classification: AUC/F1; regression: MAE/MSE) and are not directly compared against each other, each constituting a self-contained analysis of its respective modeling approach. All design choices – temporal (not random) train/validation/test splitting, class-imbalance handling, and computational trade-offs dictated by limited hardware resources – are made explicit, addressing a documentation gap present in several of the reviewed works (e.g., optimizer choices, hyperparameters and regularization details left partially unspecified).

The contributions of this work are:

- a systematic comparison between tree-based and neural tabular models for flight delay classification on a realistic, pre-departure-only feature scenario;
- a documented, reproducible preprocessing pipeline for the Aeolus tabular modality, including a concrete diagnosis and fix of a subtle LightGBM early-stopping pitfall caused by monitoring a miscalibrated metric under class weighting;
- interpretability analysis (SHAP, permutation importance) across the tabular models, cross-validated against each other;
- a sequential LSTM module on the flight-chain modality and its fusion with the tabular branch.

†Department of Physics and Astronomy, University of Padova, email:
kevin.brugnera@studenti.unipd.it,
sara.pasquato.1@studenti.unipd.it,
libero.pollini@studenti.unipd.it

The remainder of this report is organized as follows. Section II reviews related work. Section III outlines the overall processing pipeline. Section IV describes the dataset and the tabular and sequential preprocessing. Section V details the learning architectures used. Section VI reports the complete results. Section VII concludes.

II. Related Work

Datasets. Public flight-delay datasets have historically been limited in coverage – spanning short time periods, often constrained to a single airport or a small geographic region, and – with few exceptions – restricted to a flat tabular structure lacking richer modalities such as aircraft rotation sequences or resource-sharing graphs [2]. Aeolus [2] addresses these limitations with three aligned modalities (tabular, flight-chain, flight-network) and a rigorous temporal evaluation split, empirically showing that random splitting inflates AUC by 0.057 on average – a finding that directly motivated our splitting strategy (Section IV).

Sequential approaches. LATTICE [3] combines an LSTM with attention over ADS-B trajectories and a static-feature branch for arrival delay classification, showing a significant gain from the dual-branch design – the direct architectural reference for our planned fusion. McCarthy et al. [4] use a multi-hop autoregressive LSTM with transfer learning across heterogeneous airlines. Sun et al. [5] propose a three-tier framework (flight/airport/network), mirroring Aeolus's own modality structure.

Spatial/graph approaches. TrajCNN [6] and DenseNet-LSTM-FBLS [7] represent ground traffic or airport congestion as images/grids for CNN-based processing. Both Sun et al. [5] (DST-GAT) and Aeolus itself (VGAE+AFM) report only modest gains from graph-based components over sequential baselines, motivating our decision not to pursue a graph/CNN branch.

Tabular and hybrid models. Desai et al. [8] and Jha et al. [9] compare classical and hybrid (FCNN+RDF) models, confirming the competitiveness of tree-based methods over purely neural architectures on tabular flight data – a pattern also observed in our own tabular results (Section VI).

Among the reviewed works, LATTICE [3] is the only one integrating tabular and sequential information end-to-end, but at a much smaller scale and without a temporal evaluation protocol; none of the reviewed works does so on real, large-scale, multi-year flight data. This work combines the enriched-representation idea of [9] with the dual-branch architecture of [3], applied to the native structure of Aeolus.

III. Processing Pipeline

At a high level, the pipeline (Figure 1) consists of three stages: (1) data acquisition and cleaning from the raw Aeolus/BTS source files; (2) two parallel feature-engineering and modeling tracks – a tabular track (Section IV–V) and a sequential track operating on the flight-chain modality; (3) a fusion stage combining the learned representations of both tracks into a single classifier.

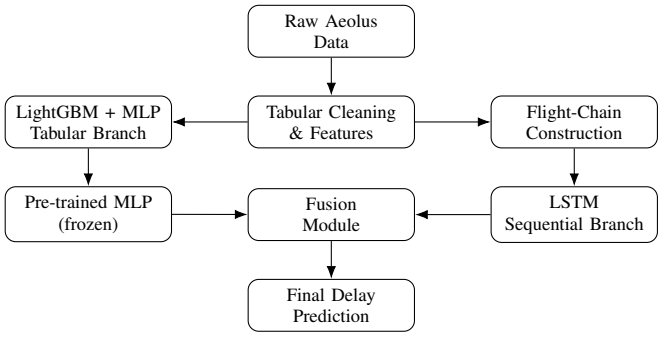


Fig. 1: Overview of the proposed flight delay prediction framework.

IV. Signals and Features

A. Dataset

Aeolus [2] integrates three aligned modalities (tabular, flight-chain, flight-network) over 2016-2024. This work uses a subset spanning 2022-2024, excluding the pandemic period (2020-2021), during which delay patterns are non-stationary. Raw data was obtained from the official Kaggle repository[1] via `kagglehub`.

For the tabular-sequential fusion (Section V), the MLP is additionally re-trained on a reduced 2022-only subset, using a different splitting strategy motivated and detailed in Section IV-C alongside the flight-chain construction that requires it.

B. Tabular preprocessing

From the raw fields, a *pre-departure* feature scenario was selected: only information available before the actual departure is used (scheduled times, weather forecasts, carrier/route identifiers), excluding post-departure fields that would make the task artificially easier. The resulting tabular set comprises 7 categorical and 14 continuous features (Table 1), plus `DEP_DELAY`/`ARR_DELAY` as targets, for a total of **19,341,439** flights after concatenating the three years.

TABLE 1: Tabular feature set (23 columns).

Type	Features
Categorical	carrier, flight number, year, month, day, origin, destination
Continuous	scheduled dep./arr. time, elapsed time, flight count, weather (temp., precip., wind) at origin/destination, origin/destination coordinates
Target	departure and arrival delay (minutes)

Additional engineered features include cyclical (sine/cosine) encodings of hour-of-day, day-of-week and month; great-circle origin-destination distance; a scheduling-based airport congestion index (flights scheduled from the same origin airport within the same 2-hour window); and `FL_WEEK` (week of year), treated as an additional categorical feature – bringing the total to 32 input features (8 categorical, 24 continuous).

[1] https://www.kaggle.com/datasets/flnny123/mfddmulti-modal-flight-delay-dataset

Following the temporal-split evidence discussed in Section II, data was split **chronologically** 60:20:20 – training up to Oct. 2023 (11,604,863 flights), validation up to Jun. 2024 (3,868,288), test up to Dec. 2024 (3,868,288). The positive class (`ARR_DELAY`>15 min) represents 20.8%/18.5%/19.5% of train/val/test respectively. Extreme delays were removed *only from the training set* (1st–99th percentile, bounds $[-36, 206]$ min, 1.86% of training flights removed), keeping validation and test representative of the true delay distribution. Categorical variables are label-encoded fitting exclusively on the training set; class imbalance is handled via class weighting rather than oversampling, for scalability at this data volume.

C. Flight-chain preprocessing

Since a flight chain (an aircraft's sequence of flights within a 24-hour window) cannot be split across train/validation/test, splitting is performed at the level of whole calendar days, stratified within each month (60/20/20 of the days in each month, fixed seed, shared with the sequential branch) - the same day-based, month-stratified strategy adopted by Aeolus itself for its flight-chain modality [2]. This choice is particularly suited to the 2022-only subset used for the fusion (Section V): with a single year available, a strict chronological cutoff would confine entire calendar months exclusively to the test set, with no training exposure to that time of year. This concern does not apply to the three-year cutoff of Section IV-B, whose training portion already spans complete annual cycles across every calendar month, making the stricter past-only-training/future-only-testing ordering both safe and preferable there. Both strategies remain clearly distinct from - and safer than - the fully unconstrained random splitting shown to inflate metrics in Section II.

The sequential experiments in this work use two distinct input representations. For the LSTM BASELINE, we rely directly on the pre-computed flight chains provided by the Aeolus dataset. These chains are downloaded from Kaggle via the `Kagglehub` module and loaded as PyTorch tensors. Each chain is a sequence of up to 6 flights operated by the same aircraft on the same day[2], identified by `(OP_CARRIER, OP_CARRIER_FL_NUM, FL_DATE)` and sorted chronologically. This direct use of the provided chains ensures comparability with the Aeolus benchmark baselines and avoids introducing additional preprocessing choices into the LSTM BASELINE.

For the LSTM+MLP fusion model, **we instead construct our own flight chains** (see `mlp_lstm_preprocessing.ipynb`) from the raw tabular data, as the Aeolus-provided chains do not include the engineered features required by the MLP branch. Starting from the cleaned tabular dataset (Section IV-B), we group flights in the same fashion, then sort each group by scheduled departure time. Each group forms a chain; we truncate or pad each chain to a fixed length of 6 time steps, dropping all flights with missing data. This yields a six-tensor dataset: dense weather features, sparse categorical features, binary delay labels, raw delay values, valid sequence lengths, and the additionalMLP-only features.

A key challenge in constructing these chains is the limited RAM of the Google Colab free tier: For this reason we were forced to process the data (over 5M flights) day-by-day: for each split, we iterate over unique flight dates, process one day's worth of flights into a TensorDataset, write it to a temporary file, and immediately free the memory. After all days are processed, we merge the per-day files into a single TensorDataset and save it to Google Drive. This chunked workflow, while computationally less efficient than an in-memory approach, was necessary to avoid out-of-memory crashes on a 12 GB runtime.

Categorical variables such as `OP_CARRIER` and `OP_CARRIER_FL_NUM` are encoded manually on the union of the 2022 and 2023 training data, then applied to the corresponding yearly datasets. The result is a sequential dataset that maintains the same split structure as the tabular branch, to be used by the fused LSTM+MLP model. From then on, LSTM takes the first five tensors, while MLP reconstructs some features using all six tensors and especially the lat one.

V. Learning Framework

This section describes the architectures adopted for both the **tabular** and **sequential** branches, concluding with their fusion.

A. Gradient-boosted trees (LightGBM)

Two LightGBM configurations were trained: a fixed-hyperparameter baseline and a version tuned via Optuna (20 trials over learning rate, `num_leaves`, `min_child_samples`). The high-cardinality feature `OP_CARRIER_FL_NUM` (>6,900 distinct values) was excluded from native categorical treatment, as it causes disproportionate histogram-construction cost in LightGBM. Class weighting (`scale_pos_weight`) is applied in the loss; as for the MLP (Section V-B), the classification threshold is selected by maximizing F1 on the validation set rather than assumed fixed at 0.5, given the generally miscalibrated probabilities produced under class weighting.

Training uses early stopping on validation AUC (patience of 30 rounds, up to 500 boosting rounds for the baseline and the tuned model), consistent with the classification metric used in Aeolus's own tabular benchmark [2].

B. Neural tabular model (MLP)

A feed-forward network processes each categorical feature through a dedicated learned embedding layer (dimension $\min(50, \lceil(\text{cardinality}+1)/2\rceil)$), concatenated with normalized continuous features, followed by two dense layers (64, 64 units) with dropout (0.4, 0.3) and a sigmoid output; L2 regularization (weight decay 10^{-3}) is applied via the Adam optimizer. The 64-dimensional penultimate representation is

[2]Note that this day-based splitting can in theory interrupt chains in case of chains spanning multiple days; however, in (US-based) Aeolus this is a rare occurrence.

exposed as a reusable embedding for the future fusion with the sequential branch, mirroring LATTICE's dual-branch design [3]. This configuration was selected after a small ablation over hidden-layer width, dropout and weight decay (Section VI).

For the tabular-sequential fusion, the MLP was additionally re-trained on the 2022-only, day-blocked subset described in Section IV-C, with an auxiliary departure-delay (`DEP_DELAY_BIN`) classification head added alongside the primary arrival-delay head, sharing the same underlying embedding, to enrich the shared representation prior to fusion (results in Section VI-B).

C. Sequential model (LSTM)

The sequential modality of the Aeolus dataset is modeled through an LSTM-based recurrent neural network branch to capture temporal dependencies within flight chains. As for the tabular branch (Section IV-C), categorical features are mapped into 4-dimensional embedding vectors and concatenated with dense features at each time step, forming the LSTM input representation. The model consists of a single-layer LSTM with hidden dimension 32. The resulting hidden representations are regularized through a Dropout layer (0.2) and processed by a fully connected **regression** head producing two outputs, corresponding to `ARR_DELAY` and `DEP_DELAY`.

A Huber loss was chosen in order to reduce the influence of outliers (especially flights that are severely delayed), defined with weights (to account for class imbalance) and masked (for the padding in chains). Training is performed on the same temporal split adopted for the MLP model (Section V-B), using AdamW optimization (learning rate 10^{-3} and weight decay 10^{-4}). The model is trained for 50 epochs with early stopping (patience of 5 rounds). Results are reported in Section VI-C.

D. Early Fusion Model

The early fusion architecture integrates tabular and sequential information before the LSTM processing stage (Section V-C). A pre-trained MLP encoder (Section V-B) is used to extract a 64-dimensional tabular embedding, with frozen parameters and disabled gradient computation during the forward pass. The tabular representation is reconstructed from flight-chain features (Section IV-C), including categorical, temporal, geographical, and weather variables. This embedding is concatenated at each time step with dense features and 4-dimensional categorical embeddings, forming the input representation for the LSTM.

The sequential model follows an identical configuration to that of Section V-C with regard to the LSTM part. The MLP encoder remains frozen, while LSTM and regression layers are jointly optimized.

E. Training monitoring

For tabular models, training and validation losses, together with AUC scores, are monitored at each epoch or boosting round to assess convergence and identify potential overfitting. Sequential models are trained using a *Masked Weighted Huber* loss to account for variable-length flight chains and delay imbalance. Since chains are padded to a fixed length of 6 flights, a mask based on the actual chain length excludes padded timesteps from the loss computation. The Huber component provides robustness to large regression errors, while delayed flights are assigned higher weights to increase their contribution during optimization.

Learning curves for both tabular and sequential models are reported in Section VI, providing an analysis of training dynamics and model convergence.

F. Interpretability

Both branches are analyzed with SHAP (`TreeExplainer` for LightGBM, a model-agnostic permutation-based explainer for the MLP) and, for the MLP, permutation feature importance, to cross-validate which features drive predictions across architecturally different models.

VI. Results

A. Tabular branch comparison

Table 2 reports test-set metrics. Both LightGBM variants outperform the standalone MLP (AUC 0.685 vs. 0.679), consistent with the tree-based dominance on tabular data reported by [8], [9]. Optuna tuning (20 trials) did not improve over the fixed-hyperparameter baseline (0.6849 vs. 0.6852 AUC, with a larger train–test gap: 0.079 vs. 0.066), suggesting the defaults were already near-optimal. All models exceed the official Aeolus tabular benchmark on the same task (best reported AUC: 0.623, AutoInt) [2].

Figures 2–4 report per-epoch/per-iteration training and validation loss and AUC. For LightGBM, validation loss rises sharply within the first $\sim$50–100 iterations and then plateaus, while validation AUC improves steadily throughout – a direct confirmation of the loss/AUC divergence under class weighting discussed in Section V. Baseline and tuned configurations show nearly identical dynamics, consistent with the small AUC gain from tuning. The MLP instead converges and plateaus within 1–2 epochs. Despite these different training dynamics, all three models converge to a similar validation AUC ceiling ($\approx$0.65–0.66), suggesting an irreducible difficulty component in the task – consistent with the stochastic delay factors (weather, ATC interventions) discussed in Section I – rather than an architecture-specific limitation.

For LightGBM, gain-based importance and SHAP (full analysis available in attached notebook) consistently rank `FL_WEEK`, `OP_CARRIER` and `ORIGIN_INDEX`/`DEST_INDEX` among the top predictors. The MLP instead favors duration/distance features (`great_circle_km`, `CRS_ELAPSED_TIME`), suggesting the two model families exploit complementary aspects of the feature set – a promising signal for the fusion.

AUC varies moderately across months (0.62–0.70) and top-volume airports (0.61–0.71) for both models, with no severe

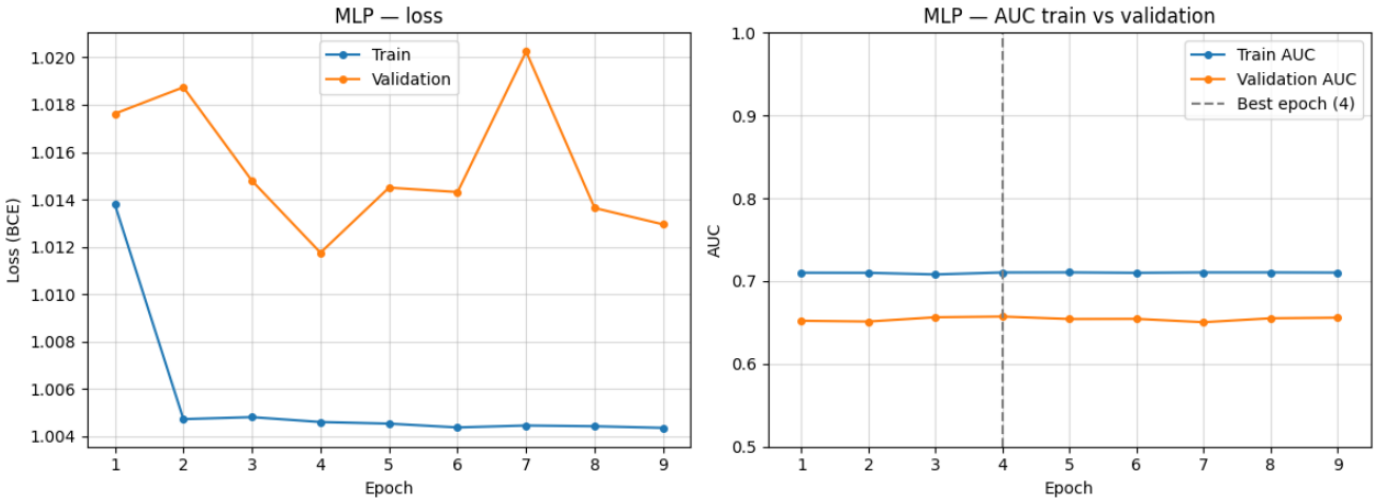


Fig. 2: MLP: training/validation loss (left) and AUC (right) per epoch.

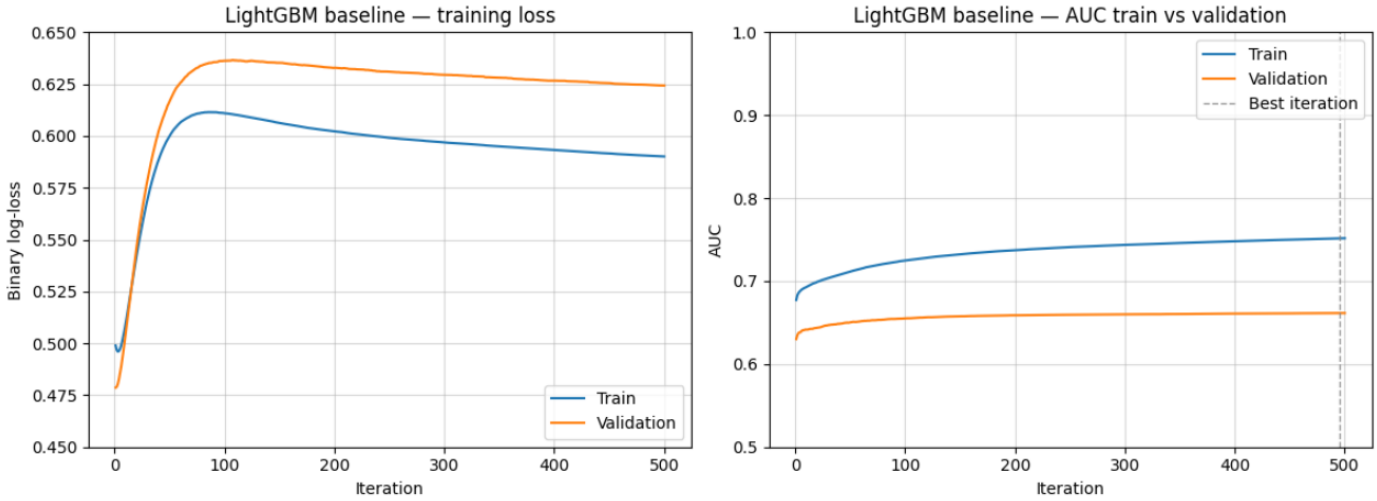


Fig. 3: LightGBM baseline: training/validation binary log-loss (left) and AUC (right) per boosting iteration.

subgroup collapse, suggesting reasonably uniform behavior across time and geography.

B. MLP results for the fusion-ready configuration (2022-only subset)

The MLP re-trained for fusion (Section V) uses a different subset and splitting strategy (day-blocked, 2022-only; Section IV-C) than the standalone analysis above, so its row in Table 2 is *not* directly comparable to the other tabular results. Training used 3,688,075 / 1,390,791 / 1,332,357 flights (train/val/test); interpretability analysis ranks `OP_CARRIER`, `FL_WEEK` and the cyclical departure-hour feature highest for both delay tasks. The resulting trained weights (shared embedding, both task heads) are used as the tabular branch in the fusion architecture (Section VI-D).

C. LSTM results on flight-chains

The LSTM baseline was trained on Aeolus-provided 2022-2023 chains. On the test set, a regression MAE of about 30 min (arrival) and 27 min (departure) was achieved (see table 3), indicating that the model captures ranking reasonably well but struggles with exact delay magnitudes. Tuning the classification threshold on validation data (see table 2) improves the F1 from 0.390 and 0.375 to approximately 0.40 for both targets, only statistical fluctuations away of the LightGBM baseline (F1 0.401). However, it should be noted that this model does not make use of any engineered features. The learning curves (Figure 5a) show a fast plateau, similar to the MLP, with hints of minor overfitting.

D. EarlyFusion model results

The LSTM+MLP fusion model achieved limited improvements over the standalone LSTM baseline (Table 2; regression metrics in Table 3). While the fusion model slightly reduced the regression MSE for both ARR (2925.0 to 2851.1) and DEP (2685.7 to 2610.3) delays, classification metrics were marginally lower, with reduced AUC, F1-score, precision,

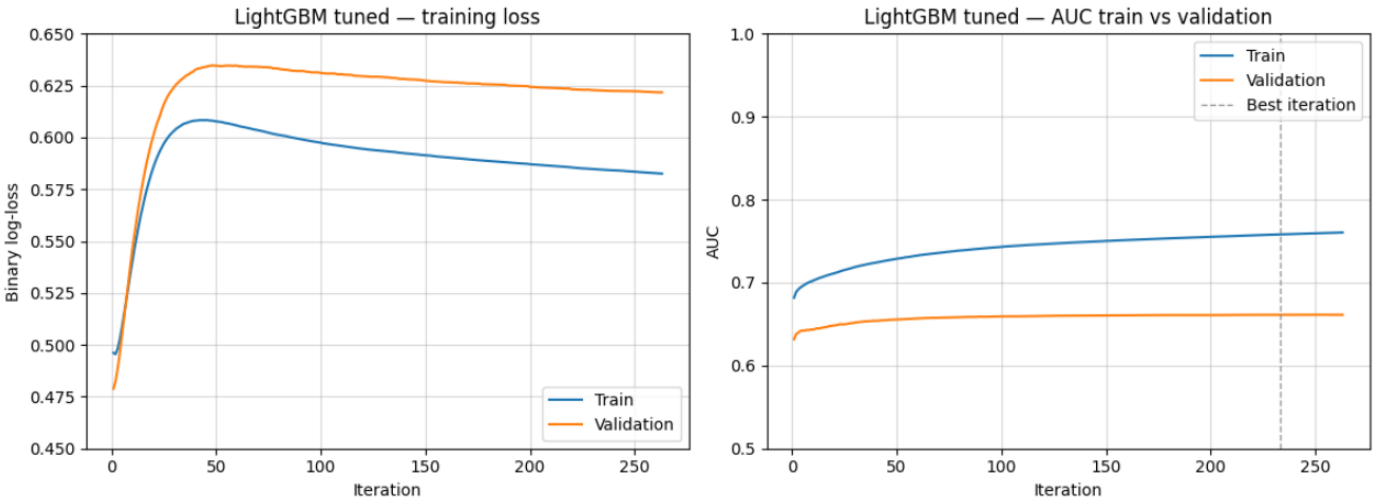
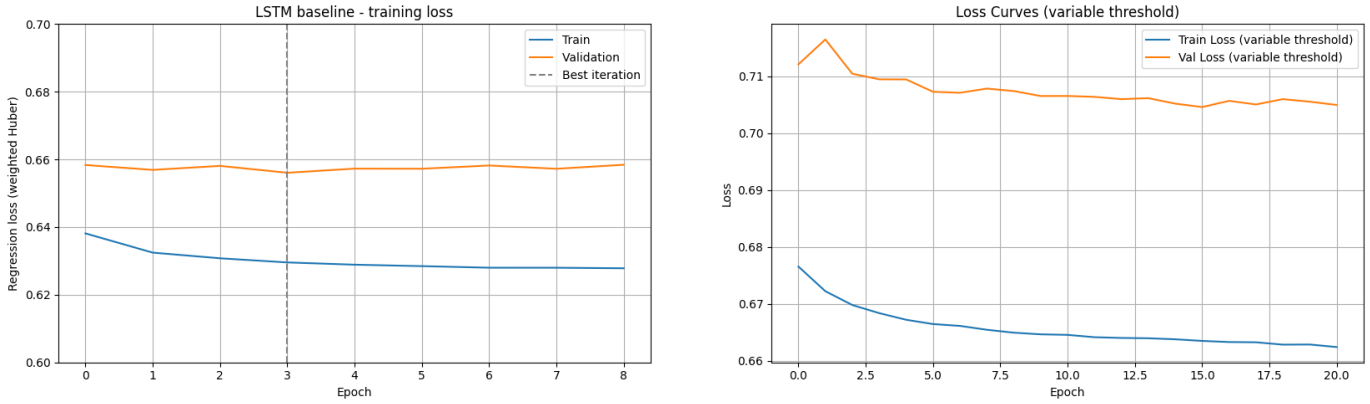


Fig. 4: LightGBM tuned: training/validation binary log-loss (left) and AUC (right) per boosting iteration.



(a) LSTM baseline

(b) LSTM+MLP

Fig. 5: Training/validation loss per epoch for the LSTM baseline (left) and the LSTM+MLP fusion model (right).

TABLE 2: Arrival classification test-set metrics for LGBM baseline, LGBM tuned, MLP and MLP for fusion; arrival prediction test-set metrics for LSTM baseline and LSTM + MLP (threshold selected on validation; "eng." stands for feature engineering)

Model	AUC	Acc.	F1	Prec.	Rec.
LGBM baseline tab. 2022-2024, eng.	0.685	0.613	0.401	0.288	0.664
LGBM tuned tab. 2022-2024, eng.	0.685	0.614	0.402	0.288	0.663
MLP tab. 2022-2023, eng.	0.679	0.644	0.400	0.298	0.607
MLP for fusion custom tab. 2022-only, eng.	0.687	0.662	0.403	0.306	0.593
LSTM baseline seq. 2022-2023, no eng.	0.681	0.646	0.399	0.296	0.612
LSTM + MLP custom tab+seq. 2022-only, eng.	0.658	0.638	0.383	0.286	0.583

and recall. The training behaviour (Figure 5b) showed a fast plateau and mild overfitting pattern similar to the standalone

TABLE 3: Regression performance (MSE and MAE) comparison between the standalone LSTM and the LSTM+MLP fusion model on the test set.

Model	**ARR**		**DEP**	
	MSE	MAE	MSE	MAE
LSTM	2925.0	**30.3**	2685.7	**27.3**
LSTM+MLP	**2851.1**	31.3	**2610.3**	28.5

LSTM (Figure 5a), suggesting that the frozen MLP embedding provided limited additional information beyond the sequential features already captured by the LSTM.

This indicates that the tabular embedding may provide complementary information for regression tasks, but its contribution is less effective for classification. Since the LSTM already processes dense and categorical flight-chain features, part of the information encoded by the MLP may overlap with the sequential representation.

VII. Concluding Remarks

This work compared gradient-boosted trees, a neural tabular model, and sequential (LSTM) architectures – standalone and fused with tabular features – on flight delay prediction. The performance of the fused model is comparable to that of the standalone LSTM, although it exhibits a slight degradation across most evaluation metrics; therefore, the additional computational and implementation complexity introduced by the multimodal framework does not justify its use for the considered task. However, further optimization strategies and experiments performed with increased computational resources may lead to improved performance and allow a more comprehensive assessment of the potential benefits of the proposed multimodal framework.

Despite markedly different training dynamics, all architectures converged to a similar AUC range (≈0.65–0.69). This suggests that the observed performance ceiling may be driven by intrinsic task difficulty, including factors such as weather variability and ATC-related decisions, rather than by limitations of a specific architecture. Therefore, when additional model complexity does not lead to a measurable improvement, simpler and more efficient approaches remain a reasonable choice, in line with Occam's Razor.

Promising extensions, however, are left for future work: using a flight's realized departure delay as an input feature for predicting its own arrival delay, then generalizing this propagation signal across the chain; training and evaluating the already-implemented late-fusion variant (`FlightChainLateFusion`); extending the fusion experiments beyond the single Colab-memory-constrained year (2022) to the full 2022-2024 span; and incorporating the airport network structure that Aeolus provides but this work did not use. Additionally and in hindsight, data preprocessing could have benefit from a standardized, parallelized pipeline.

Beyond the results, this project was a practical exercise in applied deep learning. We learned that decreasing training loss alone is not evidence of learning – comparison against trivial baselines is always essential. We also had to debug various subtler issues, which we could not have foreseen when starting the work, especially related to handling many-dimensions, large datasets. Overall, this project reinforced that meaningful progress in applied machine learning comes as much from disciplined debugging and honest evaluation as from architectural novelty.

References

[1] AirHelp, "The impact of flight disruption on the economy and environment." AirHelp Report, 2023.

[2] L. Xu, X. Yuan, Y. Liang, S. Yin, and Y. Wu, "Aeolus: A multi-structural flight delay dataset," in *39th Conference on Neural Information Processing Systems (NeurIPS 2025), Track on Datasets and Benchmarks*, 2025.

[3] A. Chaudhuri *et al.*, "Lattice: Lstm and attention for flight arrival delay prediction using trajectory and contextual data," in *SESAR Innovation Days*, 2024.

[4] N. McCarthy, M. Karzand, and F. Lecue, "Amsterdam to dublin eventually delayed? lstm and transfer learning for predicting delays of low cost airlines," in *Proceedings of the AAAI Conference on Artificial Intelligence (IAAI-19)*, pp. 9541–9546, 2019.

[5] J. Sun, T. L. E. Dijkstra, K. Aristodemou, V. S. Buzeţelu, T. Falat, T. G. Hogenelst, N. Prins, and B. C. Slijper, "Designing recurrent and graph neural networks to predict airport and air traffic network delays," in *International Conference for Research in Air Transportation (ICRAT)*, 2022.

[6] Shao *et al.*, "Trajcnn: Departure delay prediction using tarmac trajectory representation," *Neurocomputing*, 2021.

[7] Yin *et al.*, "Densenet-lstm-fbls: A spatio-temporal-fuzzy framework for flight delay severity classification," *Mathematics*, 2025.

[8] Desai, Jadhav, Salunkhe, Shaikh, and Surwase, "Flight delay prediction system using deep learning models," *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, vol. 5, no. 3, 2025.

[9] Jha, Jha, Pandey, and Babiceanu, "Flight delay prediction using hybrid machine learning approach: A case study of major airlines in the united states," *arXiv preprint arXiv:2409.00607*, 2024.